

FIT2099 Assignment 2

[REQ 1]

Name: Erwynna Soo Wen Xin

Student ID: 36555789

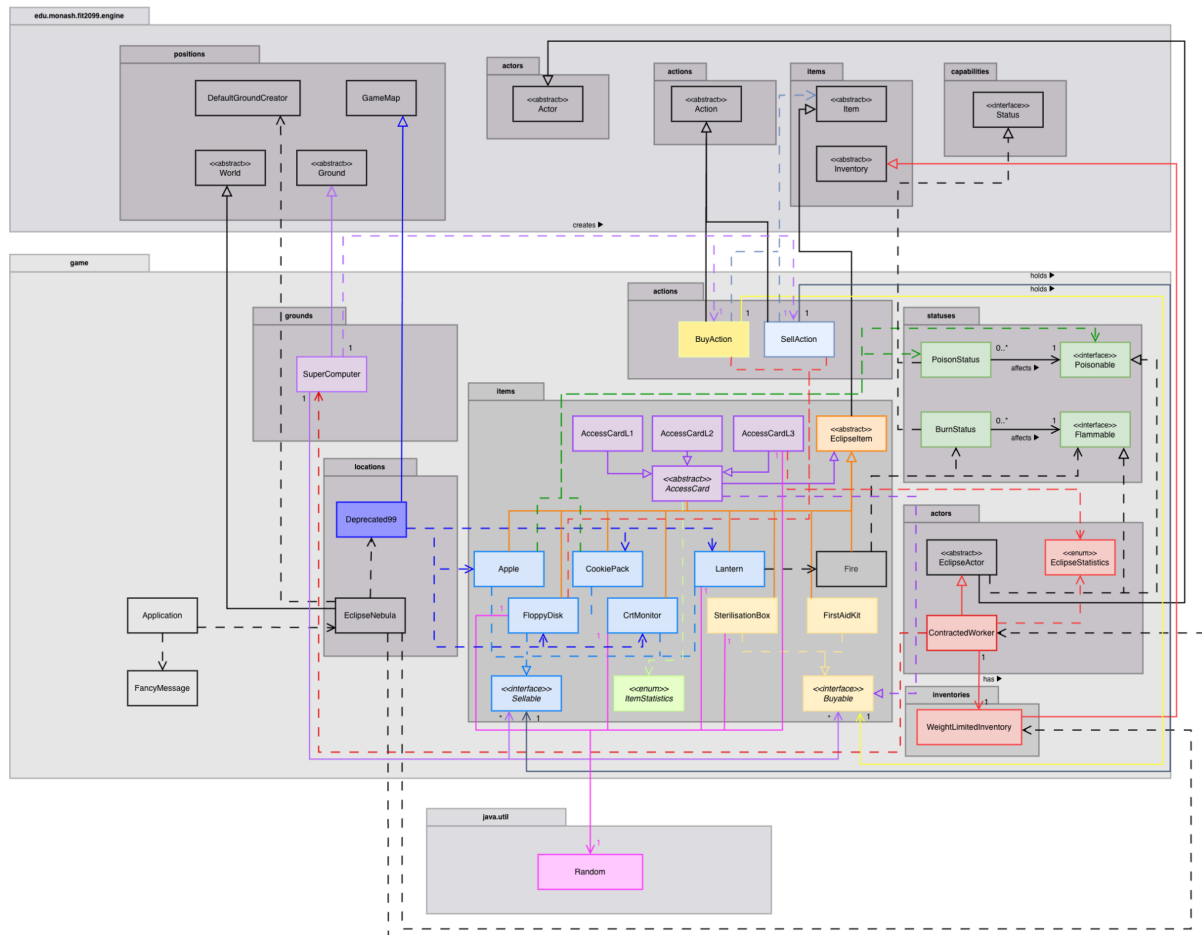
Design Diagrams

UML

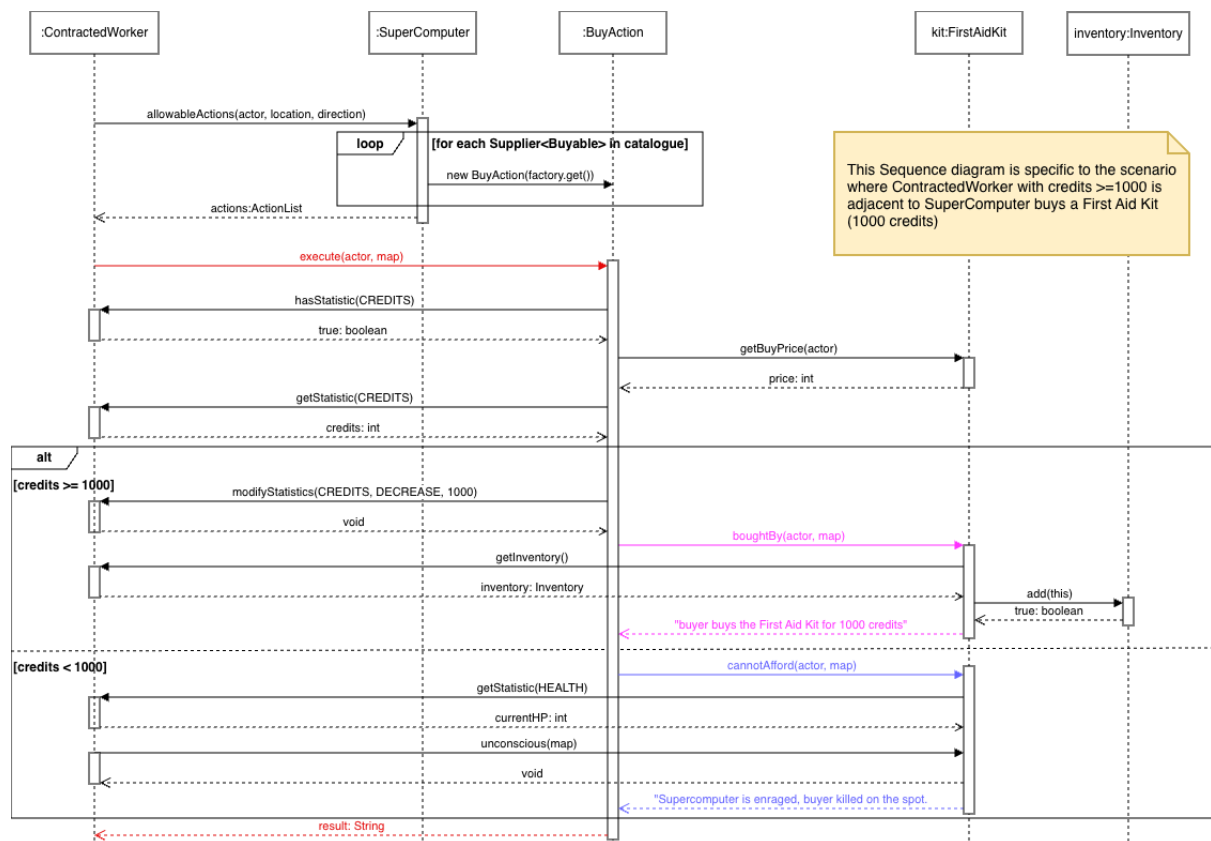
The colour grouping reflects the five major design components of REQ1:

1. Purple = Access card purchase system
2. Yellow = General buying pipeline
3. Blue = Selling/scrap pipeline
4. Red = Actor economy (credits, inventory)
5. Green = Transaction side-effects

This visual separation makes it easier to trace how each requirement feature flows through the system;
from catalogue (purple/yellow/blue) -> actions (yellow/blue) -> items (yellow/blue) -> effects (green) -> actors (red)



Sequence Diagram



REQ 1

A. SuperComputer Catalogue and the Buy/Sell Pipeline

Design 1: Catalogue Injected as a List of Suppliers (Implemented Design)

The SuperComputer holds its catalogue as a list of suppliers, each producing a fresh Buyable item on demand. The catalogue is typed against the Buyable interface, so the terminal never needs to know the concrete type of any item it stocks. This means new items can be added to the catalogue without touching the SuperComputer class at all.

Pros	Cons
<u>Decouples the terminal from concrete items</u> , so it depends only on the Buyable abstraction and can stock anything that implements it.	<u>Adds a small layer of indirection</u> , since each catalogue entry is a factory call rather than a direct field.
<u>Produces fresh instances on every purchase</u> , preventing accidental sharing of a single item reference between actors.	<u>Requires the static factory method to be kept in sync</u> whenever a new buyable item is introduced.
<u>Supports the Open-Closed Principle</u> because new items extend the catalogue without modifying SuperComputer.	<u>Uses generic supplier types</u> , which may be slightly less familiar to read than a plain item list.

Design 2: Hard-Coded Catalogue Inside the SuperComputer (Alternative Design)

In this design, the SuperComputer would internally create its own item instances inside allowableActions(), constructing each item directly with the new keyword. The catalogue would be a fixed sequence of statements that name every item class explicitly.

Pros	Cons
<u>Straightforward to read</u> because the items the terminal sells are listed inline in one place.	<u>Tightly couples the terminal to every item class</u> , so adding or removing an item forces a change to SuperComputer.java.
<u>Requires no extra types</u> such as Supplier or generic parameters.	<u>Violates the Open-Closed Principle</u> because the terminal must be reopened and edited for every new buyable.
<u>Slightly fewer files to navigate</u> during initial implementation.	<u>Makes testing harder</u> , because the catalogue cannot be swapped out for a smaller test list.

Design 1 is used because it decouples the catalogue from the terminal through dependency injection. Adding a new purchasable item only requires extending the static factory method, without modifying SuperComputer's core logic (Open-Closed Principle). In contrast, Design 2 would tightly couple the terminal to every item class, forcing edits whenever a new item is introduced and reducing the ability to test the terminal independently.

B. Buying and Selling Through Capability Interfaces

Design 1: Buyable and Sellable Capabilities with Generic Actions (Implemented Design)

The ability to be purchased or sold is treated as a capability rather than an inherent property. The Buyable interface provides the contract for purchasable items, while the Sellable interface provides the equivalent for sellable items. Each concrete item implements whichever interface applies; FirstAidKit, SterilisationBox, and AccessCard implement Buyable, while Apple, CookiePack, Lantern, FloppyDisk, and CRTMonitor implement Sellable.

BuyAction and SellAction are written against these interfaces rather than concrete item classes. BuyAction orchestrates the purchase by checking the item's price, verifying the actor has sufficient credits, deducting the cost, and delegating item-specific effects to the item's boughtBy method. This keeps the action layer generic; BuyAction never needs to know whether it is handling a FirstAidKit or an AccessCard. Item-specific behaviour lives entirely inside each item's boughtBy or soldBy method, so the action classes remain free of type checks and branching logic.

PROS & CONS of Design 1

Pros	Cons
<u>Eliminates duplicated action classes.</u> since one BuyAction handles every purchasable item and one SellAction handles every sellable item.	<u>Requires every item to implement the correct interface.</u> so introducing a new transactional item is slightly more deliberate.
<u>Encapsulates side-effects inside the item.</u> keeping the action layer thin and free of type checks.	<u>Pushes domain logic into items.</u> which means an item class can grow if its purchase behaviour is complex.
<u>Supports the Dependency Inversion Principle</u> because BuyAction and SellAction depend on abstractions rather than concrete items.	<u>Requires reading both the action class and the item class</u> to trace a complete transaction, which adds one extra file to navigate

Design 2: One Action Subclass Per Item (Alternative Design)

Every purchasable item would have its own dedicated action class (e.g., BuyFirstAidKitAction, BuySterilisationBoxAction), with item-specific logic implemented directly inside execute().

Pros	Cons
<u>Each transaction's logic</u> is visible in a single class without needing to follow a delegation chain.	<u>Causes class explosion</u> , with one new action subclass required for every new item.
<u>Items can stay as plain data</u> without implementing any extra interfaces.	<u>Duplicates the same credit-deducting and inventory-adding code</u> across every action subclass.
<u>No generic types or interface plumbing</u> are needed.	<u>Couples actions to concrete item classes</u> , violating the Open-Closed Principle and making the action layer fragile.

Design 1 is used because it eliminates duplicated action classes, with one BuyAction handling every purchasable item and one SellAction handling every sellable item. Unlike Design 2, this avoids repeating the same transaction logic across many action subclasses (DRY). Actions depend on abstractions rather than concrete item classes, which reduces the coupling between the action layer and individual items (Dependency Inversion Principle).

C. Item Side-Effects on Sale and Purchase

Design 1: Reusing the Engine Status System Through Capability Interfaces (Implemented Design)

Item side-effects (poison, burn, fire) are modelled as Status objects that integrate with the engine's existing per-turn lifecycle, so effects expire and apply damage automatically without any custom tracking code in the actor. Actors declare which effects they can receive through capability interfaces (Poisonable, Flammable). Items query whether an actor supports a given capability before applying an effect, avoiding type checks against concrete actor classes. Environmental effects such as Fire spawning on adjacent tiles are treated as ordinary items added to the map through the engine's existing item-placement system, keeping the effects model consistent across the whole codebase.

Note that item prices and side-effect values (e.g., poison duration, burn damage, spawn chances) have been declared as named constants within each item class rather than hardcoded literals scattered throughout methods. This eliminates connascence of meaning, changing it into connascence of name (weaker), improving readability and maintainability. For example, if Apple's poison effect needs adjustment, modifying a single constant ensures consistency rather than hunting through multiple method calls.

PROS & CONS of Design 1

Pros	Cons
<u>Reuses the engine's tick lifecycle</u> , so effects expire automatically without any per-turn bookkeeping inside the actor.	<u>Adds extra interfaces (Poisonable, Flammable)</u> , which the actor must implement to be a valid target.
<u>Decouples items from concrete actor types</u> by routing every effect through a capability lookup rather than a class check.	<u>Requires an effect-application path</u> for each new status, since the actor must declare the matching capability.
<u>Keeps environmental effects (fire on adjacent tiles)</u> consistent with normal item placement, avoiding a separate effects subsystem.	<u>Side-effects are spread across several small classes</u> rather than living in one place.

Design 2: Custom Per-Effect Classes Managed by the Actor (Alternative Design)

Each effect would be its own dedicated class (e.g., PoisonEffect, BurnEffect) held in a collection on the actor. The actor would iterate over this collection each turn, applying damage and reducing duration manually.

Pros	Cons
<u>Behaviour for each effect</u> lives in one self-contained class.	<u>Forces the actor to manage effect lifecycles manually</u> , duplicating logic the engine already provides through Status.
<u>Avoids the engine's status system</u> , which may feel more lightweight at first.	<u>Bypasses an engine feature</u> that exists precisely for this purpose, reducing consistency with the rest of the codebase.
<u>Effects are easy to find</u> because they all sit in the actor's effect list.	<u>Makes the actor responsible</u> for every effect type, so adding new effects forces the actor class to grow.

Design 1 is used because it reuses the engine's existing Status lifecycle, avoiding the need to reimplement per-turn tracking and cleanup in the actor (DRY). It also decouples items from concrete actor types by routing effects through capability interfaces, which makes adding new effects more straightforward. Design 2 would require the actor to manually manage every effect's duration and damage, duplicating logic the engine already handles correctly.

D. Access Card Hierarchy

Design 1: Abstract AccessCard with Tier-Specific Subclasses (Implemented Design)

AccessCard is modelled as an abstract class that implements Buyable, with three concrete subclasses (AccessCardL1, AccessCardL2, AccessCardL3). Each subclass inherits shared state (price, weight, clearance level) from the parent and overrides boughtBy to define tier-specific purchase effects. AccessCardL2 deducts health from the buyer as a blood-sample fee, and AccessCardL3 has a 50% chance to charge a hidden fee on top of the purchase price. AccessCardL1 has no side-effect and simply enters the inventory. The clearance level is exposed through a single getter, allowing REQ2 doors to check a card's tier without inspecting its runtime class.

PROS & CONS of Design 1

Pros	Cons
<u>Shares common state and behaviour</u> (price, weight, symbol, clearance statistic) through inheritance, avoiding duplication across the three card classes.	<u>Introduces a small class hierarchy</u> for what is conceptually a single concept (a card).
<u>Encapsulates each tier's quirks</u> inside its own subclass, keeping behaviour local and easy to find.	<u>Requires a new subclass</u> every time a new clearance tier is introduced.
<u>Exposes clearance level through a single <code>getClearanceLevel()</code> getter</u> backed by an <code>ItemStatistics</code> value, so other systems can read a card's tier without inspecting its runtime type.	<u>Clearance level is stored as an integer statistic</u> rather than a dedicated enum, which is slightly less type-safe but integrates cleanly with the engine's statistic system.

The inheritance hierarchy introduces three classes instead of one, but each subclass isolates tier-specific behaviour (L2's blood sample, L3's hidden fee). This avoids conditional branching inside a single `AccessCard` class, which would grow messier as new tiers are added. Each tier's logic is easier to test independently, and adding a new tier only requires creating a new subclass without modifying `SuperComputer` or `BuyAction` (Open-Closed Principle, Single Responsibility Principle).

Design 2: Single AccessCard Class Parameterised by Level (Alternative Design)

In this design, there would be just one concrete AccessCard class, and the level would be passed into the constructor as an integer. All tier-specific behaviour, such as the L2 blood sample and the L3 hidden fee, would live inside a single boughtBy method that branches on the level value.

Pros	Cons
<u>Fewer classes to maintain</u> , with all card logic centralised in one file.	<u>Centralises behaviour</u> through level branching, which grows messier as new tiers are added.
<u>Easier initial setup</u> because there is no hierarchy to design.	<u>Violates the Open-Closed Principle</u> since a new tier forces a change inside the existing class rather than an extension.
<u>Constructor-driven</u> , so creating any tier is a single new statement.	<u>Mixes concerns inside one class</u> , weakening cohesion and making each tier's behaviour harder to locate.

Conclusion of REQ1

The implemented design separates the SuperComputer transaction into small, loosely-coupled components. The terminal presents the catalogue, actions orchestrate credit movement, items handle transaction-specific effects, and the engine's status system manages effect lifetimes; each component interacts through narrow interfaces rather than concrete classes (**Single Responsibility Principle**).

The design absorbs growth through extension rather than modification. New sellable or purchasable items can be added by implementing the relevant interface and extending the factory method, without touching existing actions or the terminal. Actions depend on abstractions (*Buyable*, *Sellable*) rather than concrete item classes, meaning high-level transaction logic is never affected by changes to individual items (**Dependency Inversion Principle**). Access card tiers follow the same pattern; new tiers extend the hierarchy without modifying the shared parent or any other part of the system (**Open-Closed Principle**). The trade-off is a larger number of small types, but this keeps each component focused and the system maintainable as the game grows across future requirements.